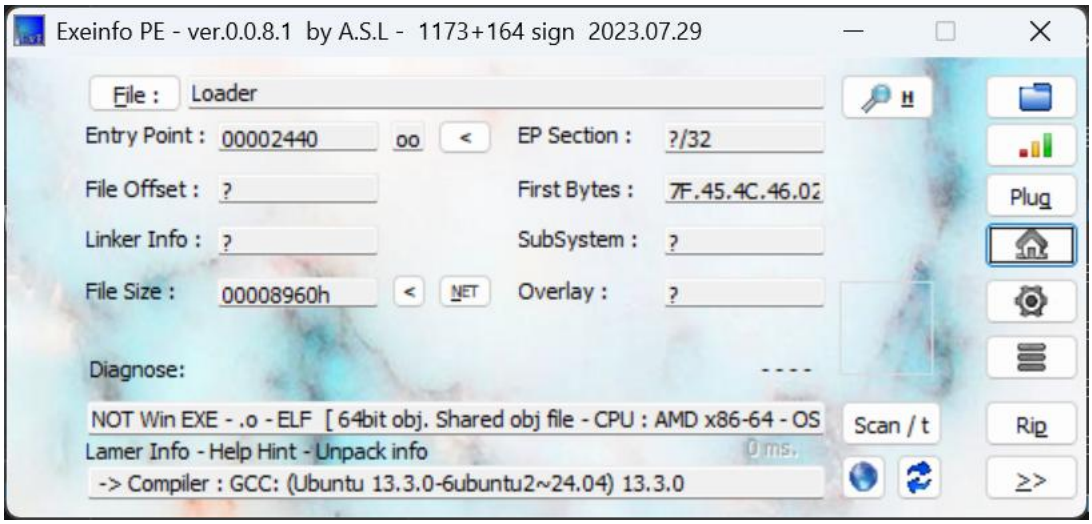
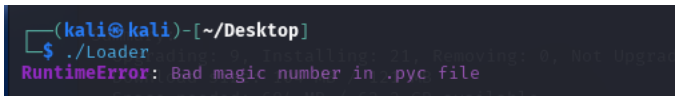


观察到 Loader 是 Linux elf 文件，执行后发现提示 Bad magic number in .pyc file



```
video.mp4
stager.pyc
stager.pyc
cannot create std::vector larger than max_size()
9*3$"
zPLR
GCC: (Ubuntu 13.3.0-6ubuntu2~24.04) 13.3.0
Scrt1.o
__abi_tag
crtstuff.c
deregister_tm_clones
__do_global_dtors_aux
completed.0
__do_global_dtors_aux_fini_array_entry
frame_dummy
__frame_dummy_init_array_entry
loader_linux.cpp
_ZStL19piecewise_construct
__FRAME_END__
__GNU_EH_FRAME_HDR
_DYNAMIC
_GLOBAL_OFFSET_TABLE_
_ZNSt12_Vector_baseIiSaIiEE12_Vector_implC2ERKS0_
_ZNSt12_Vector_baseIiSaIiEE12_Vector_implD2Ev
_ZNSt12_Vector_baseIiSaIiEE11_M_allocateEm
_ZSt10__fill_n_aIPimiET_S1_T0_RKT1_St26random_access_iterator_tag
_ZNSt7__cxx1112basic_stringIcSt11char_traitsIcESaIcEE6appendEPKcm@GLIBCXX_3.4.21
_ZNSt15__new_allocatorIcED1Ev
_Z13base64_decodeRKNSt7__cxx1112basic_stringIcSt11char_traitsIcESaIcEEE
_ZNSt12_Destroy_auxILb1EE9__destroyIPiEEvT_S3_
_edata
_ZN9__gnu_cxxneIPKcNSt7__cxx1112basic_stringIcSt11char_traitsIcESaIcEEEEEbRKNS_17__normal_iteratorIT_T0_EESE_
_ZNSt6vectorIiSaIiEED1Ev
_IO_stdin_used
_ZSt17__throw_bad_allocv@GLIBCXX_3.4
_ZNKSt7__cxx1112basic_stringIcSt11char_traitsIcESaIcEE13get_allocatorEv@GLIBCXX_3.4.21
_ZNKSt7__cxx1112basic_stringIcSt11char_traitsIcESaIcEE5c_strEv@GLIBCXX_3.4.21
__cxa_finalize@GLIBC_2.2.5
strlen@GLIBC_2.2.5
_ZNSt6vectorIiSaIiEEixEm
main
_ZSt4endlIcSt11char_traitsIcEERSt13basic_ostreamIT_T0_ES6_@GLIBCXX_3.4
_ZSt20__throw_length_errorPKc@GLIBCXX_3.4
_ZNSt12_Vector_baseIiSaIiEED1Ev
__dso_handle
_ZNSt22__uninitialized_fill_nILb1EE15__uninit_fill_nIPimiEET_S3_T0_RKT1_
payload_encoder_pyc_base64
_ZNSt7__cxx1112basic_stringIcSt11char_traitsIcESaIcEED1Ev@GLIBCXX_3.4.21
_ZNSt12_Vector_baseIiSaIiEE17_Vector_impl_dataC2Ev
_ZN9__gnu_cxx17__normal_iteratorIPKcNSt7__cxx1112basic_stringIcSt11char_traitsIcESaIcEEEEppEv
DW.ref.__gxx_personality_v0
_ZNSt12_Vector_baseIiSaIiEE12_Vector_implD1Ev
_ZSt6fill_nIPimiET_S1_T0_RKT1_
_Z17run_python_scriptRKNSt7__cxx1112basic_stringIcSt11char_traitsIcESaIcEEE
_ZNKSt7__cxx1112basic_stringIcSt11char_traitsIcESaIcEE4sizeEv@GLIBCXX_3.4.21
_fini
__libc_start_main@GLIBC_2.34
_ZSt3minInERKT_S2_S2_
_ZNSt12_Vector_baseIiSaIiEE13_M_deallocateEPim
_ZNSt14basic_ofstreamIcSt11char_traitsIcEE7is_openEv@GLIBCXX_3.4
_ZNSt6vectorIiSaIiEED2Ev
_ZSt28__throw_bad_array_new_lengthv@GLIBCXX_3.4.29
_ZNSt12_Vector_baseIiSaIiEE12_Vector_implC1ERKS0_
_ZNSt6vectorIiSaIiEE17_S_check_init_lenEmRKS0_
system@GLIBC_2.2.5
_ZNSt7__cxx1112basic_stringIcSt11char_traitsIcESaIcEE9push_backEc@GLIBCXX_3.4.21
_ZStlsIcSt11char_traitsIcESaIcEERSt13basic_ostreamIT_T0_ES7_RKNSt7__cxx1112basic_stringIS4_S5_T1_EE@GLIBCXX_3.4.21
_ZStplIcSt11char_traitsIcESaIcEENSt7__cxx1112basic_stringIT_T0_T1_EEPKS5_RKS8_
_ZSt12__str_concatINSt7__cxx1112basic_stringIcSt11char_traitsIcESaIcEEEET_PKNS6_10value_typeENS6_9size_typeES9_SA_RKNS6_14allocator_typeE
stat@GLIBC_2.33
chmod@GLIBC_2.2.5
_ZdlPv@GLIBCXX_3.4
_ZNKSt7__cxx1112basic_stringIcSt11char_traitsIcESaIcEE5beginEv@GLIBCXX_3.4.21
_ZNSt15__new_allocatorIiED2Ev
_ZNSt12_Vector_baseIiSaIiEE17_Vector_impl_dataC1Ev
_ZStlsISt11char_traitsIcEERSt13basic_ostreamIcT_ES5_PKc@GLIBCXX_3.4
_Znwm@GLIBCXX_3.4
_ZNSolsEPFRSoS_E@GLIBCXX_3.4
```

我们从 Loader 中提取 stager.pyc

```
┌──(kali㉿kali)-[~/Desktop]
└─$ strings ./Loader | grep -E '^[A-Za-z0-9+/]{200,}' > b64.txt

┌──(kali㉿kali)-[~/Desktop]
└─$ base64 -d b64.txt > stager.pyc

┌──(kali㉿kali)-[~/Desktop]
└─$ grep -a -o -P 'UEsDBBQAAAAI[A-Za-z0-9+/]{200,}' ./Loader > b64.txt

┌──(kali㉿kali)-[~/Desktop]
└─$ hexdump -C stager.pyc | head
00000000  42 0d 0d 0a 00 00 00 00  4a e6 59 69 5a 09 00 00  |B.......J.YiZ...|
00000010  e3 00 00 00 00 00 00 00  00 00 00 00 00 05 00 00  |................|
00000020  00 40 00 00 00 73 7a 00  00 00 64 00 64 01 6c 00  |.@...sz...d.d.l.|
00000030  6d 01 5a 01 01 00 64 00  64 02 6c 02 5a 02 64 00  |m.Z...d.d.l.Z.d.|
00000040  64 02 6c 03 5a 03 64 00  64 02 6c 04 5a 04 64 00  |d.l.Z.d.d.l.Z.d.|
00000050  64 02 6c 05 5a 06 64 00  64 02 6c 07 5a 07 64 00  |d.l.Z.d.d.l.Z.d.|
00000060  64 03 6c 08 6d 08 5a 08  01 00 64 0e 64 09 64 0a  |d.l.m.Z...d.d.d.|
00000070  84 01 5a 09 65 0a 64 0b  6b 02 72 76 64 0c 5a 0b  |..Z.e.d.k.rvd.Z.|
00000080  65 03 6a 0c a0 0d 65 0b  a1 01 72 6c 65 09 65 0b  |e.j...e...rle.e.|
00000090  83 01 01 00 6e 0a 65 04  a0 0e 64 0d a1 01 01 00  |....n.e...d.....|
```

反编译 stager.pyc 得到源码

```
┌──(ctf-env)─(kali㉿kali)-[~/Desktop]
└─$ uncompyle6 stager.pyc > stager.py

┌──(ctf-env)─(kali㉿kali)-[~/Desktop]
└─$ cat stager.py
# uncompyle6 version 3.9.3
# Python bytecode version base 3.7.0 (3394)
# Decompiled from: Python 3.13.2 (main, Feb  5 2025, 01:23:35) [GCC 14.2.0]
# Embedded file name: Payload_To_PixelCode_video.py
# Compiled at: 2026-01-03 23:02:18
# Size of source mod 2**32: 2394 bytes
from PIL import Image
import math, os, sys, numpy as np, imageio
from tqdm import tqdm

def file_to_video(input_file, width=640, height=480, pixel_size=8, fps=10, output_file='video.mp4'):
    if not os.path.isfile(input_file):
        return
    file_size = os.path.getsize(input_file)
    binary_string = ""
    with open(input_file, "rb") as f:
        for chunk in tqdm(iterable=(iter((lambda: f.read(1024)), b'')),
          total=(math.ceil(file_size / 1024)), unit="KB", desc="读取文件"):
            binary_string += "".join(((f"{byte:08b}") for byte in chunk))

    xor_key = "10101010"
    xor_binary_string = ""
    for i in range(0, len(binary_string), 8):
        chunk = binary_string[i[:i + 8]]
        if len(chunk) == 8:
            chunk_int = int(chunk, 2)
            key_int = int(xor_key, 2)
            xor_result = chunk_int ^ key_int
            xor_binary_string += f"{xor_result:08b}"
        else:
            xor_binary_string += chunk

    binary_string = xor_binary_string
    pixels_per_image = width // pixel_size * (height // pixel_size)
    num_images = math.ceil(len(binary_string) / pixels_per_image)
    frames = []
    for i in tqdm((range(num_images)), desc="生成视频帧"):
        start = i * pixels_per_image
        bits = binary_string[start[:start + pixels_per_image]]
        if len(bits) < pixels_per_image:
            bits = bits + "0" * (pixels_per_image - len(bits))
        img = Image.new("RGB", (width, height), color="white")
        for r in range(height // pixel_size):
            row_start = r * (width // pixel_size)
            row_end = (r + 1) * (width // pixel_size)
            row = bits[row_start[:row_end]]
            for c, bit in enumerate(row):
                color = (0, 0, 0) if bit == "1" else (255, 255, 255)
                x1, y1 = c * pixel_size, r * pixel_size
                img.paste(color, (x1, y1, x1 + pixel_size, y1 + pixel_size))

        frames.append(np.array(img))

    with imageio.get_writer(output_file, fps=fps, codec="libx264") as writer:
        for frame in tqdm(frames, desc="写入视频帧"):
            writer.append_data(frame)


if __name__ == "__main__":
    input_path = "payload"
    if os.path.exists(input_path):
        file_to_video(input_path)
    else:
        sys.exit(1)

# okay decompiling stager.pyc
```

该脚本将输入文件（称为 payload）逐字节异或 0xAA，转为二进制串，然后将每个比特映射为一个 8x8 的黑白块，最终生成 640x480 的视频文件。题目提供的 video.mp4 正是由这个脚本生成的，因此反向操作即可还原出原始的 payload。

Exp：

要读取 video.mp4 的每一帧，对每个 8x8 块取中心像素，黑色记为 1，白色记为 0，得到比特串。

将比特串每 8 位转为一个字节，并与 0xAA 异或，得到原始数据 payload.bin。

解析 payload.bin 中的 MD5 指针表，通过预计算单字节 MD5 还原 flag。

```
import hashlib
import re
import struct
import cv2
```

```
import numpy as np
from pathlib import Path

VIDEO_FILE = Path("~/Desktop/video.mp4").expanduser()
OUTPUT_BIN = Path("~/Desktop/payload.bin").expanduser()
FRAME_WIDTH = 640
FRAME_HEIGHT = 480
BLOCK_SIZE = 8
XOR_MASK = 0xAA
class VideoBitExtractor:
    def __init__(self, video_path):
        self.cap = cv2.VideoCapture(str(video_path))
        if not self.cap.isOpened():
            raise IOError(f"无法打开视频文件: {video_path}")
        self.total_frames = int(self.cap.get(cv2.CAP_PROP_FRAME_COUNT))
        self.blocks_per_row = FRAME_WIDTH // BLOCK_SIZE
        self.blocks_per_col = FRAME_HEIGHT // BLOCK_SIZE
        self.bits_per_frame = self.blocks_per_row * self.blocks_per_col

    def __iter__(self):
        frame_count = 0
        while True:
            ret, frame = self.cap.read()
            if not ret:
                break
            gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
            bits = []
            for row in range(self.blocks_per_col):
                y_center = row * BLOCK_SIZE + BLOCK_SIZE // 2
                for col in range(self.blocks_per_row):
                    x_center = col * BLOCK_SIZE + BLOCK_SIZE // 2
                    pixel = gray[y_center, x_center]
                    bits.append('1' if pixel < 128 else '0')
            frame_count += 1
            if frame_count % 20 == 0:
                print(f"[进度] 已处理 {frame_count}/{self.total_frames} 帧")
            yield ''.join(bits)
        self.cap.release()

    def close(self):
        self.cap.release()

def bits_to_payload(bit_stream):
    payload = bytearray()
```

```
    total_bits = len(bit_stream)
    valid_bits = total_bits - (total_bits % 8)
    for i in range(0, valid_bits, 8):
        byte_str = bit_stream[i:i+8]
        byte_val = int(byte_str, 2)
        payload.append(byte_val ^ XOR_MASK)
    return bytes(payload)

class PayloadParser:
    DATA_OFFSET = 0x3000
    RODATA_OFFSET = 0x2000
    PTR_COUNT = 42
    PTR_TABLE_OFFSET = DATA_OFFSET + 0x20
    PTR_SIZE = 8

    def __init__(self, payload_bytes):
        self.data = payload_bytes

    def extract_flag(self):
        md5_map = {hashlib.md5(bytes([c])).hexdigest(): chr(c) for c in range(256)}
        flag_chars = []
        for i in range(self.PTR_COUNT):
            ptr_pos = self.PTR_TABLE_OFFSET + i * self.PTR_SIZE
            ptr_value = struct.unpack_from('<Q', self.data, ptr_pos)[0]
            str_offset = self.RODATA_OFFSET + (ptr_value - 0x2000)
            end = str_offset
            while end < len(self.data) and self.data[end] != 0:
                end += 1
            md5_hex = self.data[str_offset:end].decode('ascii', errors='ignore')
            ch = md5_map.get(md5_hex, '?')
            flag_chars.append(ch)
        return ''.join(flag_chars)

def main():
    print("=== CTF 解题脚本 - 视频隐写提取 ===")

    if not VIDEO_FILE.exists():
        print(f"错误：视频文件不存在 {VIDEO_FILE}")
        return

    print("[1] 正在解析视频文件...")
    extractor = VideoBitExtractor(VIDEO_FILE)
    all_bits = []
    for frame_bits in extractor:
```

```
        all_bits.append(frame_bits)
    extractor.close()

    full_bit_string = ''.join(all_bits)
    print(f"    共提取 {len(all_bits)} 帧，总比特数: {len(full_bit_string)}")

    print("[2] 正在转换比特流为 payload 数据...")
    payload_data = bits_to_payload(full_bit_string)
    OUTPUT_BIN.parent.mkdir(parents=True, exist_ok=True)
    OUTPUT_BIN.write_bytes(payload_data)
    print(f"    已写入 {OUTPUT_BIN}，大小: {len(payload_data)} 字节")

    print("[3] 正在解析 payload 中的隐藏信息...")
    parser = PayloadParser(payload_data)
    flag_candidate = parser.extract_flag()

    match = re.search(r'dart\{[^}\r\n]+\}', flag_candidate)
    if match:
        print("\n[+] 成功找到 Flag:")
        print(match.group(0))
    else:
        print("\n[-] 未找到 Flag，可能格式不正确")
        print("    提取到的字符串为:")
        print(flag_candidate[:200] + "..." if len(flag_candidate) > 200 else flag_candidate)

if __name__ == '__main__':
    main()
```

最终 flagdart{2ab1fb8a-b830-45e7-8830-66c7e3b3e05a}